

MTRX2700 Mechatronics 2

Lab 2: Programming a microcontroller in C

Prof. Stefan B. Williams

27th March 2026

Version: First Release

Prof. Stefan B. Williams

MTRX2700 Mechatronics 2

Lab 2: Programming a microcontroller in C

Latest revision: 27th March 2026

Contributors: Stephany Berrio, Stewart Worrall

The University of Sydney

School of Aerospace, Mechanical and Mechatronic Engineering

The University of Sydney, Mechanical Engineering Building, J07

Sydney, NSW 2006

Abstract

In weeks 6 to 8 of this unit of study, we will learn how to write programs for a microcontroller using the C programming language. The tasks will be similar to the ASMLab, but the focus will be on creating c-style modules - encapsulating complexity in the '.c' file implementation, leaving the header file to describe the high level interface for users. We will also look at using interrupts instead of polling as the approach to interact with the different microcontroller submodules such that we can handle many things at the same time. We also begin developing device drivers to interact with a variety of sensors, actuators and other subsystems on the microcontroller.

1	Objectives	5
2	Pre-Lab Preparation	5
3	Assessment process	5
	3.1 Required documentation	6
	3.2 The day of the assessment.	7
	3.3 Assessment day checklist	7
4	Marking scheme	7
5	Help Getting Started	8
6	Features of all exercises	8
7	Exercises	9
	7.1 Exercise 1: Digital I/O	9
	7.1.1 Objectives	9
	7.1.2 Tasks	9
	7.2 Exercise 2: Timer Interface	10
	7.2.1 Objectives	10
	7.2.2 Tasks	11
	7.3 Exercise 3: Serial interface	12
	7.3.1 Objectives	12
	7.3.2 Tasks	12
	7.4 Exercise 4: I2C Sensor Interfacing	14
	7.4.1 Objectives	14
	7.4.2 Tasks	14
	7.5 Exercise 5: Integration Task.	15
	7.5.1 Objectives	15
	7.5.2 Tasks	15

1 Objectives

In this lab you will get a practical understanding of the theoretical concepts from this course. We will be using the C language to create programs that can:

- a) Understand how the microcontroller handles memory (pointers, iterators)
- b) Demonstrate the use of interrupts for communicating over serial
- c) Demonstrate the use of timers to perform time critical functions, including generating Pulse Width Modulation (PWM) signals
- d) Demonstrate the use of I2C serial interfaces for communicating with sensors
- e) Perform an integration of software modules.

2 Pre-Lab Preparation

Before attending the lab (you should have done most of this before Assignment 1)

- a) Complete the [online lab induction](#)
- b) Read [What is git?](#)
- c) Create a [free github account](#)
- d) Clone the [MTRX2700-2026 git repository](#)
- e) Download and install [STM32CubeIDE](#)
- f) Make sure you can log into the [Ed forum](#)
- g) Read the lab materials in this [handout](#)

3 Assessment process

This assessment is to be completed in groups of 3 or 4 students. To help facilitate the group work, this problem should be approached by having one or more students responsible for

each module (area/topic) - though it is important that each group member understands how all modules work and can describe the details to a tutor.

A good approach to achieve this is for each student to focus on a task, do some research on the topic and present their findings to the group. The group can provide feedback, and learn more about the topic themselves in the process. Before completing the task, the software plan should be completed where you write an outline of how the task/module will be completed (at this point, you should write function templates), including documentation for the inputs and outputs of each function.

Testing is an important part of software engineering. Ideally, the testing plan should come before writing any code. What are the valid range of inputs/outputs that should be handled by each function? What are the constraints and limitations? You should get a different person to be involved in the construction of a testing plan for each software module.

3.1 Required documentation

The documentation required for this project should be incorporated into the group git repository. This will allow the tutors to access the code/documentation, and provide a history of the project including who contributed to each component.

The git repository should include the following parts which are used for the assessment process:

- **The code:** the folder structure of the repo should be logical and should be able to be described to the tutor.
- **The documentation:** should be presented as the README for the group repo. This is the only documentation that will be considered during the assessment process. The README should be viewed through the github web interface for the demonstration. This README should be treated as the documentation required if someone else wants to use this code, and you are not there to be able to explain it in person. This should include the following details:
 - The details about the project (group members, roles, responsibilities)
 - High level information about the code - the function, how the program is broken down into modules.

- Instructions for the user
- Details about testing procedures
- **The minutes:** including agendas from the group meetings. Stored in a folder in the repo called 'minutes'

3.2 The day of the assessment

During the lab in week 8, each group will be required to present their solution. The tutor will be looking through the git repository which must be completed and ready to go before the demonstration. Each member of the group will be called on to describe various aspects of each module with a focus on the assessment discussion points listed at the end of each exercise.

Before the demonstration, your group must tag the repo for release.

3.3 Assessment day checklist

- ☐ Git repo documentation complete and ready for assessment
- ☐ Create a release version tag called "assessment demonstration"
- ☐ Group members prepared to describe the software modules and testing plan
- ☐ Plan how you will be live demonstrating the functions of each software module to the tutor.

4 Marking scheme

Marks will be allocated based on the following criteria

- a) Does it meet the specification
- b) Quality of the solution
 - 1) Code documentation

2) Code/algorithm structure

3) Plan for testing

c) Presentation and discussion

There will be approximately 15% allocated to each of the 5 exercises, of which approximately half will be awarded for the completeness of the solution and another half for the quality of demonstration and explanation. An additional 25% of the marks will be awarded for the overall coherence of your demonstration to the tutors, the organisation of your git repository, management of the group, distribution of tasks, completion of meeting minutes and agendas (which must be included in the repository in a folder called “minutes”) and clarity/layout/quality of the project README documentation.

5 Help Getting Started

There are code snippets and projects available as examples for performing various tasks. [This repository is linked here](#), but is also accessible from the canvas page. The first step is to clone this project on your local computer (or pull the changes if you already have the repository as there have been some changes since the start of the ASMLab). From there, you can import the projects in the repository within the IDE by selecting File/Import/General/Import existing projects, and then import the root folder of the downloaded repository.

There are a number of projects available in this repository that were described (and demonstrated) in detail during the live lecture/tutorial sessions in weeks 4 and 5. Please review these lectures if you are not clear on the helpful example projects that have been provided for you (and are very relevant to this assessment task). The mini-lectures in weeks 4 and 5 will also help in understanding what is required to write C code for the STM32 microcontroller.

6 Features of all exercises

For each exercise in this lab, the expectations are that you consider the following design principles, which will form part of the assessment process.

- Modules should perform one task

- Encapsulate complexity
- Well defined interface (how do people interact with the module)
- Design for reuse/extendability (minimise global or static variables/functions)
- Consider how you would test, or show functionality for each module (how to know if it is initialised properly?)

You should try to use function pointers where appropriate to allow a generic interface that can also be used as a connection between modules.

This lab is to be completed during your lab sessions in weeks 6 and 7. The outcomes are to be presented during your allocated lab session in week 8.

7 Exercises

The following exercises are designed to help you explore the STM32 assembly language instructions and functionality. You will be making extensive use of examples which can be accessed from the following [github repository](#).

7.1 Exercise 1: Digital I/O

7.1.1 Objectives

- Create a module that can interface to the LEDs and button specifically for the STM32F3 discovery board.
- Demonstrate the principles of software design with your implementation (see Section 6).

7.1.2 Tasks

- Basic functionality:** Create a module that allows you to set up any of the GPIO pins for input/output and provides access functions that allow you to read or write from a specific GPIO pin.

- b) Building on the generic GPIO module, create independent modules that encapsulate interfaces to the digital I/O on the Discovery Board (button and LED).
- c) Allow the passing of a function pointer to the software module on initialisation that is called (as a callback) when there is a button press.
- d) Incorporate the LED state into the module such that the only way to access it is through the header file using get/set functions.
- e) **Advanced functionality:** restrict the speed at which the LEDs can change. You will need to use a timer for this, this should be designed in a way that does not affect how long it takes to return from the set LED function (as in, it should not be a polling delay inside the set LED function).

Hints and discussion points

- What would happen if many different software modules can make changes to the LEDs?
- What would happen if the button handling callback function takes a long time to complete?
- What checks do you need to consider when setting up or using a GPIO pin for input or output?

7.2 Exercise 2: Timer Interface

7.2.1 Objectives

- Create a module that can interface to the hardware timers.
- Use the hardware timers to implement a Pulse Width Modulation (PWM) driver for a hobby servo.
- Demonstrate the software design principles with your implementation (see Section 6).

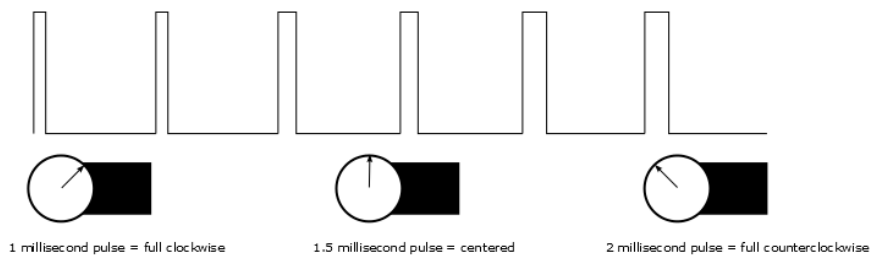


Figure 1: The third pin of the servo connector carries the control signal, used to tell the motor what position it should be in. This control signal is a specific type of pulse train. The pulses occur at a 20 mSec (50 Hz) interval, and vary between 1 and 2 mSec in width. Common servos rotate over a range of 90° as the pulses vary between 1 and 2 mSec – they should be at the center of their mechanical range when the pulse is 1.5 mSec.

7.2.2 Tasks

- a) **Basic functionality:** implement a timer software module that can trigger a callback function (using function pointers) at a regular interval. The interval should be passed as a parameter during the initialisation of the module.
- b) Implement a function that can reset the timer with a new period. The period should not be accessible outside the module scope, use get/set functions to allow outside modules to change the period.
- c) **PWM Signal Generation:** Use the basic timer functionality and GPIO module developed in Question 1 to create a timer that triggers at 50Hz (20ms period) and drives a given output pin high. The software module should then pull the output pin low at a time set by the desired position of a hobby servo. The hobby servo position is set by the width of the trigger signal, with a 1ms pulse setting the servo to the full clockwise position, 1.5ms in the middle and 2ms to the counterclockwise position as illustrated in Figure 1.
- d) **Advanced functionality:** implement a function for the module that can trigger a one-shot event (as in, timer gets reset, elapses and triggers callback then does not continue. This function should take as an input the delay in milliseconds, and the callback function pointer.

Hints and discussion points

- The example code **W06-C-timers** will be very helpful for this task.

- How much would be required to make this software module able to use different timers - as in, could you have multiple regular intervals, or multiple one-shot delays at the same time?
- What happens if one software module sets a delay, and before it elapses a different software module also tries to set a delay
- How would you drive multiple servo motors using your timer and PWM modules?

7.3 Exercise 3: Serial interface

7.3.1 Objectives

- Create a module that can interface to a serial port. It should provide functions that can be used to send or receive data contained in structures. It should also provide an interface to allow strings to be written out to a serial port for debugging purposes.
- Demonstrate the software design principles with your implementation (see Section 6).

7.3.2 Tasks

- Basic functionality:** Create a module that can interface with the UART to send an array of bytes from memory or receive an array of bytes given a known structure of the message. You can base your design on the project [W06-UART-modular-design](#) and insights gained from the Assembly lab.
- Debug functionality:** Create a `sendString` function that can be passed a pointer to a string and can transmit this on a serial port. This can be sent over the USART1 interface to allow your program to be debugged on a console on your computer.
- Send functionality:** The `sendMsg` function should take a pointer to a block of memory (defined as a structure) and a size (you can use `sizeof` to calculate the size of your structure) and should assemble this into an appropriate message structure and transmit this over a given serial port. Your function should be able to send different message types with different structures of data and you may wish

to include a message identifier to help with interpreting the received data. The function should assemble a start bit, message size, message type, message body, stop bit and checksum as a data packet before sending.

- d) **Receive functionality:** The `receiveMsg` function should read in the incoming bytes into a memory buffer until the terminating character is received. It should then check that the checksum is correct before calling a callback function (set when initialising the module) for the completion of receiving data. When the message has finished being received (based on the terminating character), the callback should be triggered including passing a pointer to the memory block received and the number of bytes received as parameters to the callback.
- e) Replace the serial receiving process with an interrupt based approach. Consider data being sent as complete when a terminating character is received.
- f) **Advanced functionality:** use an interrupt when transmitting characters. Also, use a [double buffer](#) to allow the serial to continue receiving characters while a different function can use the string.

Hints and discussion points

- How do you handle the case when there is more data received than will fit in the buffer?
- How do you determine when the incoming data has finished being received? Do you use a terminating character? What if this byte is missed or if the same byte appears elsewhere in the received data?
- What are some potential advantages and disadvantages of passing structures and raw bytes rather than strings as we did in the Assembly lab?
- How do other software modules interact with the received data? Can they request the latest data? What happens with the incoming memory buffer after someone has requested the data? Do you clear the buffer? Do you have more than one buffer?.
- What happens if someone requests new serial port data before the current stream of data is complete (i.e. before the stream is terminated)?
- What if someone keeps working on the serial rx buffer at the same time as new

data comes in?

- Remember, you should have a known working microcontroller program (in assembly) that can transmit and receive bytes over UART. This can be used to provide insights in the development of your C/interrupt driven approach.

7.4 Exercise 4: I2C Sensor Interfacing

7.4.1 Objectives

- Create a module that can set up and read data from the STM32 Discovery Board's built-in compass module via its I2C interface.
- Create a structure to store the compass data so it can be shared with other modules in your code.

7.4.2 Tasks

- a) Basic functionality:** implement an I2C interface to the STM32 Discovery Board's compass module.
- b)** Create a structure that can store the values being returned by the magnetometer so it can be used by other parts of the code or passed over a serial interface. The structure should include the raw data as well as the decoded heading and should also include a timestamp indicating the time that the data was received.
- c) Advanced functionality:** The Discovery board also includes an accelerometer (on the same I2C device) and a MEMS gyro (available on a Serial Peripheral Interface (SPI)). You may wish to create a full attitude reference estimator that estimates the heading, roll and pitch of the board.

Hints and discussion points

- The example code included in the [STM32-disco-BSP code](#) module in the repository includes code for both the I2C interface and the Magnetometer which may be helpful for this task.
- Do your compass readings correspond to the cardinal directions (N-E-S-W)? How

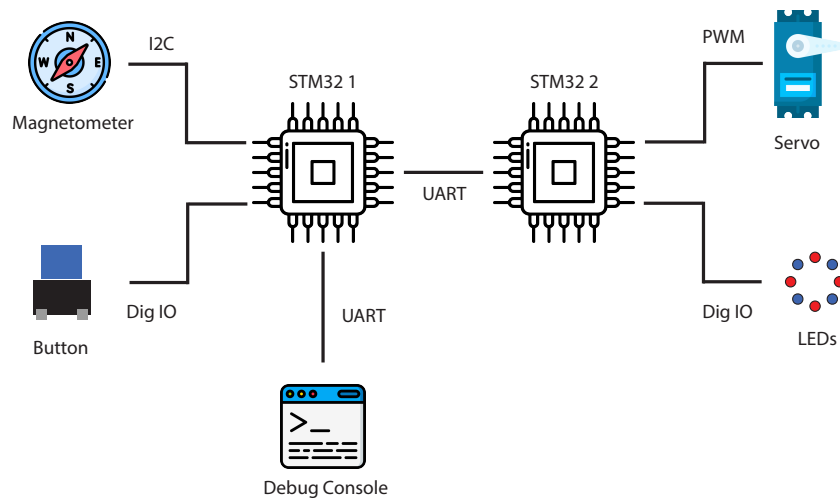


Figure 2: An overview of the integration task showing one STM32 board being used to interface to the magnetometer and button. Messages passed to a second STM32 board will drive the position of a hobby servo or the LED array.

might you calibrate your sensor to give more accurate readings of the actual heading of the device?

7.5 Exercise 5: Integration Task

7.5.1 Objectives

- Create an application that uses your modules developed during the first four exercises.
- Demonstrate the software design principles with your implementation (see Section 6).

7.5.2 Tasks

The integration task should be able to complete the following tasks:

- Using the modules developed already (any changes to the modules should be reflected back into the original projects), develop a program that can read the

magnetometer data from one STM32 board. This data should be packaged into an appropriate structure and transmitted over a UART to a second board.

- b) The first board should also have an interrupt driven function to respond to a button push. This should set a signal (either directly in the magnetometer structure or as a separate message type) that tells the second board how to display the output.
- c) The second board should be waiting to receive the messages from the first board. It should change the position of the servo in response to the heading data (noting that the servo will typically have a range of approximately 90° rather than 360° but you can scale the position appropriately. When the button is pressed, it should change to showing the heading using the array of LEDs by lighting up one of the LEDs corresponding to the heading read from the magnetometer.

Hints and discussion points

- You can debug parts of this exercise using the serial connection to the PC. Putty (and other serial terminal programs) have the ability to both send and receive data or you can connect the console in the IDE to show serial data being sent as strings to USART1.